



BASIC · INTERMEDIATE · ADVANCED

Playwright Practical Handbook

প্লেরাইট প্র্যাকটিক্যাল হ্যান্ডবুক (Python) — ইংরেজি ও বাংলা

From zero browser automation to a production-grade test suite through real problem-solving: login flows, dynamic pages, file uploads, network mocking, API testing, Page Objects, tracing and CI. Every concept: English first, then বাংলা. No filler — labs, mistakes, exercises with answers.

7 Modules

7 Real-World Labs

60+ Runnable Examples

Exercises + Answers

Python · pytest

English + বাংলা

Learn by typing, breaking & fixing

2026 Edition

Contents • সূচিপত্র

Module 00 — Start Here: Setup & How Playwright Thinks

0.1 Who This Book Is For · 0.2 What Playwright Is · 0.3 Install & First Script · 0.4 Codegen & Headed Mode · 0.5 How to Practice

Module 01 — Locators & Actions: Finding and Driving the Page

Role-based locators · filtering & chaining · fill/click/check/select · keyboard · Lab: login + inventory automation

Module 02 — Auto-Waiting & Web-First Assertions

Why sleep() is a bug · expect() assertions · actionability · waiting for URLs & load states · Lab: dynamic loading page

Module 03 — Auth, State & Awkward Real-World Pages

storage_state login reuse · uploads & downloads · dialogs · popups & new tabs · iframes · Lab: login once, test many

Module 04 — Network Control & API Testing

route() mocking · blocking & modifying · expect_response · APIRequestContext · hybrid UI+API · Lab: mock a flaky payment API

Module 05 — pytest Integration & Page Object Model

pytest-playwright fixtures · conftest & parametrize · Page Objects done right · auth fixtures · Lab: refactor a suite to POM

Module 06 — Production: Tracing, Parallelism, CI & Debugging

Trace viewer · screenshots & video on failure · parallel + retries · GitHub Actions · Capstone: e-commerce smoke suite

Appendix — Cheat Sheet, Error Decoder & Next Steps

One-page cheat sheet · decoder for the 10 errors you will actually meet · ten graded projects · the roadmap after this book

Start Here: Setup & How Playwright Thinks

শুরু এখানেই: সেটআপ ও প্লেরাইট যেভাবে ভাবে

IN THIS CHAPTER · এই অধ্যায়ে

Who this book is for · what Playwright is (and is not) · install & first script · codegen and headed mode · the 20-minute practice rule.

0.1 Who This Book Is For & How to Read It

This book assumes you already know basic Python (variables, functions, dicts, `with`, classes — roughly Modules 1–5 of the *Python Practical Handbook*) and takes you from **zero browser automation** to a **production-grade test suite**: login flows, dynamic pages, file uploads, mocked networks, API checks, Page Objects, traces and CI. No filler theory — every chapter ends in code you could actually ship to a QA pipeline.

Each chapter follows the same skeleton: **concept** → **mechanism** → **real example** → **hands-on lab** → **common mistakes** → **practice exercises with answers** → **summary**. Every section is English first, then a বাংলা panel that restates the core idea in short, natural Bangla (a rewrite, not a word-for-word translation).

★ KEY POINT

Do not just read. **Run every example against a real browser in headed mode first**, watch what it does, break it on purpose, and fix it. Seeing the browser act → fail → pass is how automation actually enters your brain.

বাংলা

এই বইটি ধরে নেয় আপনি Python-এর বেসিক জানেন, আর এখান থেকে আপনাকে শূন্য ব্রাউজার-অটোমেশন থেকে প্রোডাকশন-মানের টেস্ট স্যুট পর্যন্ত নিয়ে যাবে — লগইন ফ্লো, ডাইনামিক পেজ, ফাইল আপলোড, নেটওয়ার্ক মকিং, Page Object, ট্রেস ও CI। প্রতিটি সেকশন আগে ইংরেজি, তারপর সংক্ষিপ্ত বাংলা। সবচেয়ে গুরুত্বপূর্ণ নিয়ম: **প্রতিটি উদাহরণ প্রথমে headed মোডে (ব্রাউজার দেখা যায় এমনভাবে) চালান**, ইচ্ছা করে ভাঙুন, তারপর ঠিক করুন — এভাবেই শেখা হয়।

0.2 What Playwright Is & Where It Is Used

Plain-language first: Playwright is a library that lets your Python code drive a real browser — open pages, click buttons, fill forms, read text — exactly like a very fast, very reliable human tester.

Think of a remote control for Chrome, Firefox and WebKit (Safari's engine): your script presses the buttons, the browser does the work. It is built by Microsoft, and it dominates modern end-to-end (E2E) testing because of one core design decision: **auto-waiting** — every action automatically waits until the element is ready, killing the flaky `sleep()`-and-pray style that made older tools (Selenium-era suites) miserable. Beyond testing, the same skills power web scraping, PDF generation, screenshot services, and RPA-style business automation.

Tool era	How it waits	Result
Manual driving (Selenium 3-style code)	You add explicit waits / sleeps everywhere	Flaky tests, random failures
Playwright	Every action auto-waits for actionability	Stable, fast, debuggable

Playwright হলো এমন একটি লাইব্রেরি যা দিয়ে আপনার Python কোড সত্যিকারের ব্রাউজার চালায় — পেজ খোলে, বোতাম চাপে, ফর্ম পূরণ করে, লেখা পড়ে — অনেকটা ব্রাউজারের রিমোট কন্ট্রলের মতো। Chrome, Firefox, WebKit — তিনটিই চলে। এর সবচেয়ে বড় শক্তি **অটো-ওয়াটিং**: প্রতিটি অ্যাকশন নিজে থেকেই এলিমেন্ট প্রস্তুত হওয়া পর্যন্ত অপেক্ষা করে, তাই পুরোনো টুলের মতো `sleep()` দিয়ে অনুমান করতে হয় না। টেস্টিং ছাড়াও স্ক্র্যাপিং, স্ক্রিনশট, PDF তৈরি, অফিস-অটোমেশনে একই দক্ষতা কাজে লাগে।

0.3 Installing Playwright & Running Your First Script

Inside a fresh virtual environment (one project, one venv — always), install the pytest plugin: it brings Playwright itself plus test fixtures you'll use from Module 05. Then download the browser binaries once:

TERMINAL

```
mkdir pw-course && cd pw-course
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install pytest-playwright
playwright install chromium # downloads the browser (~150 MB)
```

Create `first.py` — the **sync API** reads top-to-bottom like normal Python (there is also an async API; learn sync first):

PYTHON

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False) # False = watch it work
    page = browser.new_page()
    page.goto("https://playwright.dev/python")
    print(page.title()) # read data back into Python
    page.get_by_role("link", name="Docs").click() # drive the page
    print(page.url)
    browser.close()
```

OUTPUT

```
Fast and reliable end-to-end testing for modern web apps | Playwright Python
https://playwright.dev/python/docs/intro
```

The object hierarchy you will live in: **Browser** (the program) → **Context** (an incognito profile: cookies, storage — cheap to create, perfectly isolated) → **Page** (one tab). Tests get a fresh context each, so they can never leak state into each other.

⚠ WARNING

Never name your own file `playwright.py` or `test.py` — Python will import your file instead of the real library and you will get baffling errors like `ImportError: cannot import name 'sync_playwright'`. (Same shadowing trap as naming a file `random.py`.)

নতুন venv-এ `pip install pytest-playwright` দিন, তারপর `playwright install chromium` দিয়ে একবার ব্রাউজার নামান। sync API সাধারণ Python-এর মতো উপর থেকে নিচে পড়া যায়। স্তরগুলো মনে রাখুন: **Browser** (প্রোগ্রাম) → **Context** (ইনকগনিটো প্রোফাইল — কুকি/স্টোরেজ আলাদা, বানানো সম্ভা) → **Page** (একটি ট্যাব)। প্রতিটি টেস্ট নিজের Context পায় বলে একটির ডেটা আরেকটিতে ছড়ায় না। সতর্কতা: নিজের ফাইলের নাম কখনো `playwright.py` রাখবেন না।

0.4 Codegen: Let Playwright Write the First Draft

`codegen` opens a browser and records your clicks as Python code — the fastest way to discover the right locators on an unfamiliar site. Treat its output as a *rough draft* you clean up, never as finished code:

TERMINAL

```
playwright codegen https://www.saucedemo.com    # record actions as Python
playwright codegen --target python-pytest URL    # emit pytest-style test
```

বাংলা

`playwright codegen URL` একটি ব্রাউজার খোলে এবং আপনার ক্লিকগুলো Python কোডে রেকর্ড করে — অচেনা সাইটে সঠিক লোকেটর খুঁজে পাওয়ার দ্রুততম উপায়। তবে এর আউটপুট **খসড়া** — পরিস্কার করে নিজের গঠনে সাজাবেন, হুবহু কপি করে প্রোডাকশনে দেবেন না।

0.5 How to Practice: the 20-Minute Rule

- ✓ **Watch first, headless later.** Develop with `headless=False` (or `pytest --headed`); switch to headless only when green.
- ✓ **Predict before you run.** Say out loud what the browser will do, then run. Wrong guess = the exact spot you're about to learn something.
- ✓ **The 20-minute rule:** when a locator won't match or a wait times out, struggle alone for 20 minutes (open the trace, inspect the page), then search.
- ✓ **Practice on safe targets:** `saucedemo.com`, `the-internet.herokuapp.com`, `demo.playwright.dev/todomvc` — built to be automated. Respect real sites' terms and robots.txt.
- ✓ **Read errors bottom-up:** the last lines name the timeout and show exactly which locator was still waiting.

বাংলা

অনুশীলনের নিয়ম: আগে `headless=False`-এ ব্রাউজার দেখে দেখে শিখুন; সবুজ হলে headless করুন। চালানোর আগে অনুমান করুন কী হবে। আটকে গেলে ২০ মিনিট নিজে চেষ্টা করুন — এরর মেসেজ নিচ থেকে পড়ুন, কোন লোকেটর অপেক্ষায় ছিল তা লেখা থাকে। অনুশীলনের জন্য বানানো সাইট (saucedemo, the-internet) ব্যবহার করুন; আসল সাইটের নিয়মকানুন মেনে চলুন।

Locators & Actions: Finding and Driving the Page

লোকেটর ও অ্যাকশন: পেজে খুঁজে বের করা ও চালানো

IN THIS CHAPTER · এই অধ্যায়ে

User-facing locators (get_by_role first) · filtering & chaining · fill/click/check/select_option · keyboard & mouse
· Lab: automate a login and product page end to end.

1.1 Locators: Point at Things the Way a User Would

Plain-language first: a locator is a *saved description* of how to find an element — "the button named Login" — not the element itself. Playwright re-finds it fresh at every action, so it survives page re-renders.

The priority order that professional suites enforce — most user-visible first, brittle CSS last:

PYTHON

page.get_by_role("button", name="Add to cart")	# 1) role + accessible name — the default choice
page.get_by_label("Password")	# 2) form fields by their label
page.get_by_placeholder("Search products")	# 3) inputs with placeholder text
page.get_by_text("Order complete")	# 4) non-interactive text
page.get_by_test_id("checkout-total")	# 5) data-testid your team controls
page.locator("css=price >> nth=0")	# last resort: raw CSS/XPath — brittle

Why roles win: `get_by_role("button", name="Login")` keeps working when designers change classes, IDs and markup — it breaks only when the *user-visible* page changes, which is exactly when a test *should* break. Roles come from the accessibility tree: `button`, `link`, `textbox`, `checkbox`, `heading`, `row`, `option`.

★ KEY POINT

A locator is **lazy**: creating it touches nothing. Only actions (`.click()`) and assertions execute a search. That's why you can build locators up-front, pass them around, and store them on Page Object classes (5.3).

বাংলা

লোকেটর হলো এলিমেন্ট খুঁজে পাওয়ার একটি **সংরক্ষিত বর্ণনা** — "Login নামের বোতামটি" — এলিমেন্ট নিজে নয়; প্রতিটি অ্যাকশনের সময় নতুন করে খোঁজা হয়, তাই পেজ রি-রেন্ডার হলেও কাজ করে। অগ্রাধিকার: আগে `get_by_role` (ব্যবহারকারী যা দেখে), তারপর label/placeholder/text/test_id, সবার শেষে কাঁচা CSS — ওটা ভঙ্গুর। রোল-ভিত্তিক লোকেটর ডিজাইন বদলালেও টিকে থাকে; ভাঙে কেবল তখনই যখন ব্যবহারকারীর চোখে পেজ সত্যিই বদলায়।

1.2 Filtering, Chaining & Strict Mode

Real pages have twenty "Add to cart" buttons. Narrow down by **chaining** (search inside a region) and **filtering** (keep matches that contain something):

PYTHON

```
row = page.get_by_role("row").filter(has_text="Sauce Labs Backpack")
row.get_by_role("button", name="Add to cart").click() # the right one of 20

card = page.locator(".product-card").filter(
    has=page.get_by_role("heading", name="Router AC1200")) # filter by child element

items = page.get_by_role("listitem")
print(items.count()) # how many matched
print(items.nth(2).inner_text()) # third item
print(items.all_inner_texts()) # -> list[str] for Python-side checks
```

⚠ WARNING

Strict mode is a feature, not a bug. If a locator matches 2+ elements, actions raise `strict mode violation` instead of guessing. Don't silence it with `.first` reflexively — refine the locator so it describes exactly one thing. `.first` that "fixes" a test usually hides a wrong target.

বাংলা

বাস্তব পেজে একই নামের বোতাম বিশটা থাকে। `.filter(has_text=...)` দিয়ে সঠিক সারি/কার্ড বেছে নিন, তারপর তার ভেতরে চেইন করে বোতামটি ধরুন। একটি লোকেটর একাধিক এলিমেন্টে মিললে Playwright ইচ্ছা করেই এরর দেয় (strict mode) — অনুমান করে ভুল জায়গায় ক্লিক করার চেয়ে এটা ভালো। `.first` দিয়ে চুপ করানোর আগে ভাবুন: লোকেটরটাই আরও নিখুঁত করা যায় কি না।

1.3 Actions: Fill, Click, Check, Select

PYTHON

```
page.get_by_label("Username").fill("standard_user") # clears, then types – use this
page.get_by_label("Comments").press_sequentially("slow typing", delay=80) # only for autocomplete UIs

page.get_by_role("button", name="Login").click()
page.get_by_role("button", name="Menu").click(force=True) # skip actionability – rare, last resort

page.get_by_role("checkbox", name="Remember me").check() # idempotent: safe if already checked
page.get_by_label("Sort by").select_option("price-desc") # <select> by value
page.get_by_label("Sort by").select_option(label="Price (high to low)")

page.get_by_role("textbox", name="Search").press("Enter") # single keys / chords: "Control+A"
page.get_by_text("Product image").hover() # reveal tooltips / menus
page.get_by_label("Upload invoice").set_input_files("invoice.pdf") # more in 3.2
```

Every action first performs **actionability checks** — visible, stable (not animating), enabled, not covered — retrying until they pass or the timeout (default 30s) expires. That is the auto-waiting promise from 0.2, and it is why Module 02 will ban `time.sleep()` outright.

বাংলা

`fill()` আগেরটা মুছে নতুন মান বসায় — ফর্মে এটাই ব্যবহার করুন; অক্ষরে-অক্ষরে টাইপ (`press_sequentially`) কেবল অটোকমপ্লিট-জাতীয় UI-তে। `check()` আগে থেকে টিক থাকলেও নিরাপদ; `select_option()` ড্রপডাউনের জন্য। প্রতিটি অ্যাকশনের আগে Playwright নিজেই যাচাই করে: এলিমেন্ট দৃশ্যমান, স্থির, সক্রিয় ও ঢাকা নয় — না হওয়া পর্যন্ত অপেক্ষা করে। `force=True` এই যাচাই এড়িয়ে যায় — শেষ অস্ত্র, অভ্যাস নয়।

1.4 Reading Data Back into Python

PYTHON

```
total_text = page.get_by_test_id("checkout-total").inner_text() # "Total: $32.39"
price = float(total_text.split("$")[1]) # normal Python takes over

href = page.get_by_role("link", name="Invoice").get_attribute("href")
is_disabled = page.get_by_role("button", name="Pay").is_disabled()
rows = page.get_by_role("row").all_inner_texts() # scrape a table -> list[str]
```

The division of labour: Playwright *extracts*, plain Python *computes* (your string kit: `split`, `strip`, `float()`). But for *verifying* page content, prefer `expect()` assertions (2.2) — they retry; a plain `assert` on `inner_text()` checks one instant.

বাংলা

`inner_text()`, `get_attribute()`, `all_inner_texts()` দিয়ে পেজের ডেটা Python-এ আনুন, তারপর চেনা স্ট্রিং-কৌশলে (`split`, `float()`) হিসাব করুন। তবে পেজের বিষয়বস্তু **যাচাই** করতে হলে ২.২-এর `expect()` ব্যবহার করুন — ওটা বারবার চেষ্টা করে; সাধারণ `assert` কেবল এক মুহূর্তের ছবি দেখে।

1.5 Lab: Automate Login + Cheapest-Product Report

Real scenario: QA asks for a nightly script that logs into the demo shop, sorts products by price, and reports the cheapest item and the cart badge count after adding it. Only chapter-1 tools:

PYTHON

```
# shop_report.py – login, sort, extract, act
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch()
    page = browser.new_page()

    page.goto("https://www.saucedemo.com")
    page.get_by_placeholder("Username").fill("standard_user")
    page.get_by_placeholder("Password").fill("secret_sauce")
    page.get_by_role("button", name="Login").click()

    page.locator("[data-test=product-sort-container]").select_option("lohi") # price low->high

    first_card = page.locator(".inventory_item").first # cheapest after sort
    name = first_card.locator(".inventory_item_name").inner_text()
    price = first_card.locator(".inventory_item_price").inner_text()
    first_card.get_by_role("button", name="Add to cart").click()

    badge = page.locator(".shopping_cart_badge").inner_text()
    print(f"Cheapest: {name} at {price} | cart badge: {badge}")
    browser.close()
```

OUTPUT

```
Cheapest: Sauce Labs Onesie at $7.99 | cart badge: 1
```

Notice the shape: **navigate → act → extract → act → extract**, with zero waits written by hand — every `fill` / `click` waited for the page by itself. Here `.first` is *correct*, not a hack: after sorting, "the first card" is precisely the thing we mean.

বাস্তব দৃশ্য: প্রতি রাতে স্ক্রিপ্ট ডেমো শপে লগইন করবে, দাম অনুযায়ী সাজাবে, সবচেয়ে সস্তা পণ্যের নাম-দাম বের করবে, কার্টে যোগ করে ব্যাজ সংখ্যা জানাবে। গঠনটা দেখুন: **যাও → করো → পড়ো → করো → পড়ো** — হাতে লেখা কোনো অপেক্ষা নেই; প্রতিটি অ্যাকশন নিজেই অপেক্ষা করেছে। এখানে `.first` সঠিক ব্যবহার — সাজানোর পরে "প্রথম কার্ড"-ই আমাদের লক্ষ্য।

1.6 Common Mistakes

Mistake	Symptom	Fix
CSS selectors for everything	Suite breaks on every redesign	<code>get_by_role</code> / <code>get_by_label</code> first
Ignoring strict-mode violations with <code>.first</code>	Test clicks the wrong element, passes anyway	Refine with <code>.filter()</code> / chaining
<code>get_by_text("Log")</code> partial match surprise	Matches "Logout" too	<code>get_by_text("Log in", exact=True)</code>
Storing <code>inner_text()</code> early, asserting later	Stale value; race conditions	Extract at the moment of use, or <code>expect()</code>
<code>force=True</code> to "fix" a failing click	Clicks through overlays users can't click through	Find why it's covered; close the overlay
XPath copied from DevTools	Unreadable, breaks weekly	Any user-facing locator

1.7 Practice Exercises (with Answers)

- Checkbox toggler:** on `the-internet.herokuapp.com/checkboxes`, ensure both checkboxes end up checked, no matter their starting state.
- Table scrape:** on `/tables`, print each customer's last name and due amount as a Python dict.
- Dropdown roundtrip:** on `/dropdown`, select Option 2, then read the selection back and `assert` it.

PYTHON

```
# --- Answers (core lines) ---
# 1 - check() is idempotent, so no if-needed logic:
for box in page.get_by_role("checkbox").all():
    box.check()

# 2 - zip two column scrapes into a dict:
names = page.locator("#table1 td:nth-child(2)").all_inner_texts()
dues = page.locator("#table1 td:nth-child(4)").all_inner_texts()
print({n: float(d.lstrip("$")) for n, d in zip(names, dues)})

# 3
dd = page.locator("#dropdown")
dd.select_option(label="Option 2")
assert dd.input_value() == "2"
```

অনুশীলন: (১) দুটি চেকবক্স যেকোনো শুরু-অবস্থা থেকে টিক করা — `check()` আগেই টিক থাকলে কিছু করে না, তাই if লাগে না; (২) টেবিল থেকে নাম ও বকেয়া তুলে Python ডিক্টে — দুই কলামের তালিকা `zip` করুন; (৩) ড্রপডাউনে Option 2 বেছে `input_value()` দিয়ে ফেরত পড়ে যাচাই। আগে নিজে চেষ্টা করুন, তারপর মেলান।

1.8 Summary

Locators are lazy, re-resolved descriptions — prefer user-facing ones (`get_by_role` above all). Narrow with `filter` / chaining instead of silencing strict mode. Actions auto-wait for actionability; `fill`, `click`, `check`, `select_option` cover 90% of driving. Extract with `inner_text` / `get_attribute`, compute in plain Python, but verify with `expect()` — coming next.

Auto-Waiting & Web-First Assertions

অটো-ওয়াইটিং ও ওয়েব-ফার্স্ট অ্যাসারশন

IN THIS CHAPTER · এই অধ্যায়ে

Why `sleep()` is always a bug · `expect()` and the retrying assertion family · negations & timeouts · URLs and load states · Lab: taming a dynamic loading page.

2.1 The Flakiness Problem & Why `sleep()` Is Always Wrong

Plain-language first: web pages change *after* they load — spinners, fetches, re-renders. A test that checks too early fails randomly; one that `sleep(5)` is slow *and* still fails on a slow day.

`time.sleep(n)` is wrong in both directions at once: too long on fast runs (waste × thousands of tests), too short on slow CI (flaky red). The professional rule is absolute: **never sleep — always wait for a condition**. Playwright gives you two mechanisms: actions auto-wait for actionability (Module 01), and assertions auto-retry until true:

PYTHON

```
import time
time.sleep(5)                                # ✗ guessing — always a bug
page.wait_for_timeout(3000)                  # ✗ same bug with a Playwright accent

from playwright.sync_api import expect
expect(page.get_by_text("Order complete")).to_be_visible() # ✓ waits for the condition
```

বাংলা

পেজ লোড হওয়ার **পরেও** বদলাতে থাকে — স্পিনার, ফেচ, রি-রেন্ডার। `sleep(5)` দুই দিকেই ভুল: দ্রুত মেশিনে সময় নষ্ট, ধীর CI-তে তবুও ফেল। পেশাদার নিয়ম একটাই: **কখনো ঘুমাবেন না — শর্তের জন্য অপেক্ষা করুন**। অ্যাকশন নিজে অপেক্ষা করে (মডিউল ১), আর `expect()` অ্যাসারশন সত্য হওয়া পর্যন্ত বারবার চেষ্টা করে — এই দুটিই যথেষ্ট।

2.2 The `expect()` Family: Assertions That Retry

PYTHON

```
from playwright.sync_api import expect

expect(page.get_by_role("heading", name="Products")).to_be_visible()
expect(page.get_by_test_id("total")).to_have_text("Total: $32.39") # exact
expect(page.get_by_test_id("total")).to_contain_text("$32")         # substring
expect(page.get_by_role("button", name="Pay")).to_be_enabled()
expect(page.get_by_label("Email")).to_have_value("a@x.com")
expect(page.get_by_role("listitem")).to_have_count(6)               # waits for exactly 6
expect(page).to_have_url("**/checkout-complete.html")              # glob patterns ok
expect(page).to_have_title("Swag Labs")

expect(page.locator(".spinner")).not_to_be_visible()                # negation retries too
expect(page.get_by_text("Saved")).to_be_visible(timeout=10_000)     # per-assertion override
```

Each of these polls the page until the condition holds or the assertion timeout (default 5s) expires — then fails with a readable diff. This is the single biggest upgrade over `assert el.inner_text() == ...`, which samples one instant and races the page.

★ KEY POINT

Memorise the split: **actions** auto-wait to *do*; **expect()** auto-waits to *verify*. If you ever feel the urge to sleep, what you actually want is one of these two — usually an `expect()` on the thing you were about to wait for.

বাংলা

`expect()` পরিবারের প্রতিটি অ্যাসারশন শর্ত সত্য না হওয়া পর্যন্ত পেজকে বারবার দেখে (ডিফল্ট ৫ সেকেন্ড), তারপর পড়ার-যোগ্য ডিফসহ ফেল করে। `to_be_visible`, `to_have_text`, `to_have_count`, `to_have_url` — এগুলো মুখস্থ করুন। ভাগটা মনে রাখুন: **অ্যাকশন** অপেক্ষা করে *করার* জন্য, **expect()** অপেক্ষা করে *যাচাইয়ের* জন্য — ঘুমানোর ইচ্ছা হলে আসলে এ দুটির একটিই দরকার।

2.3 Navigation, Load States & the networkidle Trap

PYTHON

```
page.goto("https://shop.example/checkout") # waits for the load event by default
page.get_by_role("link", name="Cart").click() # click that navigates? also handled
expect(page).to_have_url("**/cart") # then assert WHERE you are

page.go_back(); page.reload()
page.wait_for_load_state("domcontentloaded") # rarely needed explicitly
```

⚠ WARNING

`wait_for_load_state("networkidle")` is discouraged for tests: apps with polling/analytics never go idle, so it times out or lies. Assert on **visible outcomes** (an element, a URL) instead — the user doesn't care about network idleness, and neither should the test.

বাংলা

`goto()` ও নেভিগেট-করানো ক্লিক নিজেরাই লোডের অপেক্ষা করে; এরপর `expect(page).to_have_url(...)` দিয়ে **কোথায় আছেন** তা যাচাই করুন। `networkidle` এড়িয়ে চলুন — পোলিং/অ্যানালিটিক্সওয়ালা অ্যাপ কখনো "চুপ" হয় না, তাই এটা টাইমআউট করে বা মিথ্যা বলে। ব্যবহারকারী যা দেখে (এলিমেন্ট, URL) তার উপরেই অ্যাসার্ট করুন।

2.4 Timeouts: One Dial, Set Once

PYTHON

```
context = browser.new_context()
context.set_default_timeout(10_000) # actions: 10s for this whole context
page.get_by_role("button", name="Export").click(timeout=60_000) # one slow action
expect(page.get_by_text("Report ready")).to_be_visible(timeout=30_000)
```

Defaults: actions 30s, `expect()` 5s. Tune globally once (context or pytest config), override per-call only for genuinely slow operations (report generation, payment redirects). A suite full of per-call timeouts is a smell — it usually means assertions are pointed at the wrong signal.

ডিফল্ট: অ্যাকশন ৩০ সে, `expect()` ৫ সে। কনটেইনারে একবার `set_default_timeout()` দিন; সত্যিকারের ধীর কাজে (রিপোর্ট তৈরি, পেমেন্ট রিডাইরেক্ট) কেবল সেই কলটিতে override করুন। প্রতিটি কলে আলাদা timeout ছড়িয়ে থাকা মানে সাধারণত ভুল সংকেতের দিকে অ্যাসার্ট করছেন।

2.5 Lab: Taming a Dynamic Loading Page

Real scenario: a dashboard shows a spinner, fetches data, then renders the result — the exact shape of every SPA screen you will ever test. Prove the whole lifecycle with assertions only, zero sleeps:

PYTHON

```
# dynamic_lab.py – spinner appears, spinner dies, result verified
from playwright.sync_api import sync_playwright, expect

with sync_playwright() as p:
    page = p.chromium.launch().new_page()
    page.goto("https://the-internet.herokuapp.com/dynamic_loading/2")

    page.get_by_role("button", name="Start").click()

    spinner = page.locator("#loading")
    expect(spinner).to_be_visible() # 1) work started
    expect(spinner).not_to_be_visible(timeout=10_000) # 2) work finished
    expect(page.get_by_text("Hello World!")).to_be_visible() # 3) correct outcome

    print("dynamic content verified, no sleeps used")
```

OUTPUT

dynamic content verified, no sleeps used

The three-beat pattern — **started → finished → correct** — documents the user experience and survives any timing: fast local run, slow CI, throttled network. Compare with `sleep(6)`: slower on average *and* flaky. This one lab pattern removes 80% of real-world test flakiness.

বাস্তব দৃশ্য: ড্যাশবোর্ডে স্পিনার আসে, ডেটা আসে, ফল দেখায় — প্রতিটি SPA স্ক্রিনের চেহারা এটাই। তিন-তালের প্যাটার্নটি শিখুন: **শুরু হয়েছে** (স্পিনার দৃশ্যমান) → **শেষ হয়েছে** (স্পিনার অদৃশ্য) → **ফল সঠিক** (কাজক্ষিত লেখা দৃশ্যমান)। কোনো sleep নেই, তবু দ্রুত মেশিনে দ্রুত, ধীর CI-তেও সবুজ — বাস্তবের ৮০% ফ্লেকিনেস এই এক প্যাটার্নেই দূর হয়।

2.6 Common Mistakes

Mistake	Symptom	Fix
<code>time.sleep()</code> / <code>wait_for_timeout()</code>	Slow suite that still flakes	<code>expect()</code> on the awaited outcome
<code>assert loc.inner_text() == x</code>	Random failures on slow renders	<code>expect(loc).to_have_text(x)</code> — it retries
<code>is_visible()</code> in an <code>if</code> for waiting	Checks one instant, returns False, moves on	<code>expect(...).to_be_visible()</code>
<code>networkidle</code> as the "page ready" signal	Timeouts on apps that poll	Assert a visible element or URL

Asserting text before navigation settles	Sees the old page's text	<code>expect(page).to_have_url(...)</code> first
Cranking one global 60s timeout	Failures take a minute to report	Keep defaults; override the one slow call

2.7 Practice Exercises (with Answers)

- Disappearing button:** on `/dynamic_loading/1` the element exists but is hidden. Show that `to_be_visible()` vs a raw "is it in the DOM" check tell different stories.
- Add/remove counter:** on `/add_remove_elements/`, click "Add Element" 5 times and assert the delete-button count without reading `.count()` in a loop.
- URL gate:** after the saucedemo login from Lab 1.5, assert you landed on `**/inventory.html` and the Products heading is visible.

PYTHON

```
# --- Answers (core lines) ---
# 1 - attached (in DOM) vs visible are different states:
finish = page.locator("#finish")
expect(finish).to_be_attached()      # present already
expect(finish).not_to_be_visible()   # but hidden until the job ends

# 2 - to_have_count retries until 5 exist:
for _ in range(5):
    page.get_by_role("button", name="Add Element").click()
expect(page.get_by_role("button", name="Delete")).to_have_count(5)

# 3
expect(page).to_have_url("**/inventory.html")
expect(page.get_by_text("Products")).to_be_visible()
```

বাংলা

অনুশীলন: (১) DOM-এ আছে (`to_be_attached`) আর দৃশ্যমান (`to_be_visible`) — দুটি আলাদা অবস্থা, পার্থক্যটা নিজে দেখুন; (২) ৫ বার ক্লিকের পর `to_have_count(5)` — গণনা সত্য হওয়া পর্যন্ত নিজেই অপেক্ষা করে; (৩) লগইনের পরে আগে URL, তারপর হেডিং যাচাই — নেভিগেশনের পরে অ্যাসার্টের সঠিক ক্রম এটাই।

Auth, State & Awkward Real-World Pages

অথেনটিকেশন, স্টেট ও বাস্তবের কঠিন পেজ

IN THIS CHAPTER · এই অধ্যায়ে

Contexts as clean rooms · storage_state: log in once, reuse everywhere · uploads & downloads · JS dialogs · popups and new tabs · iframes · Lab: one login, many fast tests.

3.1 Contexts: One Clean Room per Test

Plain-language first: a **BrowserContext** is a fresh incognito profile — its own cookies, localStorage, sessions. Contexts are cheap (~milliseconds); browsers are expensive (~seconds). So: **one browser per run, one context per test.**

PYTHON

```
browser = p.chromium.launch()           # expensive — do once
ctx_admin = browser.new_context()       # cheap, isolated
ctx_buyer = browser.new_context()       # a second, totally separate "user"
admin, buyer = ctx_admin.new_page(), ctx_buyer.new_page()
# admin's login cookie does not exist in buyer's world — multi-user tests in one process
```

বাংলা

Context মানে একদম নতুন ইনকগনিটো প্রোফাইল — নিজস্ব কুকি, localStorage, সেশন। ব্রাউজার চালু করা ব্যয়বহুল (সেকেন্ড), context বানানো সস্তা (মিলিসেকেন্ড)। নিয়ম: **পুরো রানে একটি ব্রাউজার, প্রতিটি টেস্টে একটি context**। দুটি context মানে দুটি সম্পূর্ণ আলাদা "ব্যবহারকারী" — অ্যাডমিন-ক্রেতা একসাথে টেস্ট করা যায়, একজনের লগইন অন্যজনে ছড়ায় না।

3.2 storage_state: Log In Once, Reuse Everywhere

Logging in through the UI in every test wastes minutes per run and hammers your auth service. The professional pattern: log in **once**, save the context's cookies + localStorage to a JSON file, and start every future context pre-authenticated:

PYTHON

```
# save_auth.py — run once (or once per CI job)
page.goto("https://www.saucedemo.com")
page.get_by_placeholder("Username").fill("standard_user")
page.get_by_placeholder("Password").fill("secret_sauce")
page.get_by_role("button", name="Login").click()
expect(page).to_have_url("**/inventory.html") # prove login worked BEFORE saving
page.context.storage_state(path="auth.json")   # cookies + localStorage -> file

# every_test.py — every later context is born logged in
context = browser.new_context(storage_state="auth.json")
page = context.new_page()
page.goto("https://www.saucedemo.com/inventory.html") # straight past the login wall
```


⚠ WARNING

`auth.json` contains live session tokens — treat it like a password. Add it to `.gitignore`, never commit it, and regenerate it when sessions expire (a stale file is the #1 cause of "suddenly every test fails at goto").

বাংলা

প্রতিটি টেস্ট UI দিয়ে লগইন করা মানে প্রতি রানে মিনিট নষ্ট। পেশাদার প্যাটার্ন: **একবার** লগইন করুন, `storage_state(path="auth.json")` দিয়ে কুকি+localStorage ফাইলে রাখুন, পরে প্রতিটি context `storage_state="auth.json"` দিয়ে জন্ম থেকেই লগইন-করা অবস্থায় শুরু হবে। সতর্কতা: এই ফাইলে জীবন্ত সেশন টোকেন থাকে — পাসওয়ার্ডের মতো গোপন রাখুন, git-এ দেবেন না, মেয়াদ ফুরালে নতুন করে বানান।

3.3 Uploads, Downloads & JS Dialogs

PYTHON

```
# UPLOAD — no OS file dialog needed; set the input directly:
page.get_by_label("Choose File").set_input_files("invoice.pdf")
page.get_by_label("Photos").set_input_files(["a.png", "b.png"]) # multiple

# DOWNLOAD — declare you expect it, then trigger it:
with page.expect_download() as dl_info:
    page.get_by_role("link", name="Export CSV").click()
download = dl_info.value
download.save_as(f"reports/{download.suggested_filename}")

# DIALOGS (alert/confirm) — register the handler BEFORE the trigger:
page.on("dialog", lambda d: d.accept()) # auto-OK every confirm
page.get_by_role("button", name="Delete account").click()
```

The `with page.expect_*` shape — *arm the listener, then act* — is a core Playwright idiom; you'll meet it again for popups (3.4) and network responses (4.3). Arming *after* clicking is a race you will lose on fast machines.

বাংলা

আপলোডে OS-এর ফাইল ডায়ালগ লাগে না — `set_input_files()` সরাসরি ইনপুটে ফাইল বসায়। ডাউনলোড ও ডায়ালগে মূল ছন্দটি শিখুন: **আগে শ্রোতা প্রস্তুত করুন, তারপর ট্রিগার করুন** — `with page.expect_download():` ব্লকের ভেতরে ক্লিক। ক্লিকের পরে শ্রোতা বসালে দ্রুত মেশিনে ঘটনা আগেই ঘটে যাবে — এই রেস আপনি হারবেন। ডায়ালগ হ্যান্ডলার (`page.on("dialog", ...)`) ও ট্রিগারের আগেই নিবন্ধন করুন।

3.4 Popups, New Tabs & iframes

PYTHON

```
# A link with target=_blank opens a NEW page object:
with context.expect_page() as new_page_info:
    page.get_by_role("link", name="Terms & Conditions").click()
terms = new_page_info.value # the new tab, fully driveable
expect(terms).to_have_url("**/terms")

# IFRAMES — a page inside the page (payment widgets, editors, ads):
frame = page.frame_locator("#payment-iframe")
frame.get_by_label("Card number").fill("4242 4242 4242 4242")
frame.get_by_role("button", name="Pay").click()
```

Elements inside an iframe are invisible to normal page locators — if a locator times out on something you can clearly see, check the DevTools Elements panel for an `<iframe>` ancestor. Payment providers (Stripe, SSLCommerz) almost always live in one.

বাংলা

নতুন ট্যাব একটি নতুন `Page` অবজেক্ট — `context.expect_page()` দিয়ে ধরুন, তারপর স্বাভাবিকভাবে চালান। `iframe` হলো পেজের ভেতরে আরেকটি পেজ (পেমেন্ট উইজেট, এডিটর) — সাধারণ লোকেটর সেখানে ঢুকতে পারে না; `frame_locator()` দিয়ে ঢুকুন। চোখে স্পষ্ট দেখা যাচ্ছে অথচ লোকেটর টাইমআউট? DevTools-এ দেখুন উপরে কোনো `<iframe>` আছে কি না — পেমেন্ট প্রোভাইডাররা প্রায় সবসময় `iframe`-এ থাকে।

3.5 Lab: One Login, Many Fast Tests

Real scenario: your suite has 40 tests behind a login wall. Logging in via UI in each adds $\sim 6s \times 40 = 4$ wasted minutes per run. Restructure: one login script produces `auth.json`; every test starts pre-authenticated:

PYTHON

```
# fast_suite.py – login once, then N instant-start "tests"
from playwright.sync_api import sync_playwright, expect
BASE = "https://www.saucedemo.com"

def save_login(browser):
    page = browser.new_context().new_page()
    page.goto(BASE)
    page.get_by_placeholder("Username").fill("standard_user")
    page.get_by_placeholder("Password").fill("secret_sauce")
    page.get_by_role("button", name="Login").click()
    expect(page).to_have_url("**/inventory.html")
    page.context.storage_state(path="auth.json")
    page.context.close()

def test_cart_badge(browser):
    ctx = browser.new_context(storage_state="auth.json") # born logged-in
    page = ctx.new_page()
    page.goto(f"{BASE}/inventory.html")
    page.get_by_role("button", name="Add to cart").first.click()
    expect(page.locator(".shopping_cart_badge")).to_have_text("1")
    ctx.close()

def test_sort_persists(browser):
    ctx = browser.new_context(storage_state="auth.json")
    page = ctx.new_page()
    page.goto(f"{BASE}/inventory.html")
    page.locator("[data-test=product-sort-container]").select_option("za")
    expect(page.locator(".inventory_item_name").first).to_have_text("Test.allTheThings() T-Shirt (Red)")
    ctx.close()

with sync_playwright() as p:
    browser = p.chromium.launch()
    save_login(browser) # pay the login cost ONCE
    test_cart_badge(browser)
    test_sort_persists(browser)
    print("2 tests passed with a single UI login")
```

OUTPUT

```
2 tests passed with a single UI login
```

Architecture worth absorbing: the login is a **setup step, not a test**; each test gets a pristine context (no shared pages, no ordering coupling); and the login is *verified* (`to_have_url`) before saving — never save a state you haven't proven. In Module 05, pytest fixtures make this pattern automatic.

বাংলা

বাস্তব দৃশ্য: লগইন-দেয়ালের পেছনে ৪০টি টেস্ট — প্রতিটিতে UI-লগইন মানে প্রতি রানে ~৪ মিনিট নষ্ট। সমাধান: লগইন একটি **সেটআপ ধাপ, টেস্ট নয়** — একবার করে `auth.json` বানান; প্রতিটি টেস্ট নিজস্ব নির্মল context নিয়ে জন্ম থেকেই লগইন-করা।
লক্ষ করুন: সংরক্ষণের আগে লগইন সফল হয়েছে কি না যাচাই করা হয়েছে — যা প্রমাণ করেননি তা কখনো সংরক্ষণ করবেন না।
মডিউল ৫-এ pytest ফিক্সচার এই প্যাটার্নটাই স্বয়ংক্রিয় করবে।

3.6 Common Mistakes

Mistake	Symptom	Fix
Sharing one page across tests	Test order suddenly matters	New context per test
UI login inside every test	Minutes wasted per run	<code>storage_state</code> reuse (3.2)
Saving auth state without verifying login	Every later test fails mysteriously	<code>expect(page).to_have_url()</code> first
Committing <code>auth.json</code> to git	Leaked session tokens	<code>.gitignore</code> it; regenerate in CI
Registering <code>expect_download / on("dialog")</code> after the click	Event already fired; test hangs	Arm the listener before the trigger
Page locators for iframe content	Timeout on a visibly-present element	<code>frame_locator()</code>

3.7 Practice Exercises (with Answers)

- Two users, one process:** open two contexts on `the-internet.herokuapp.com/login` (user `tomsmith` / `SuperSecretPassword!`): log one in, prove the other is still logged out.
- Upload roundtrip:** on `/upload`, upload a file you create with Python, and assert the page echoes its filename.
- Confirm-box coverage:** on `/javascript_alerts`, click "JS Confirm" twice — accept once, dismiss once — and assert the result text differs.

PYTHON

```
# --- Answers (core lines) ---
# 1 – isolation is the assertion:
expect(page_a.get_by_text("You logged into a secure area!")).to_be_visible()
page_b.goto("https://the-internet.herokuapp.com/secure")
expect(page_b.get_by_text("You must login")).to_be_visible() # b never logged in

# 2
from pathlib import Path
Path("hello.txt").write_text("hi")
page.locator("#file-upload").set_input_files("hello.txt")
page.get_by_role("button", name="Upload").click()
expect(page.locator("#uploaded-files")).to_have_text("hello.txt")

# 3 – swap the handler between clicks:
page.once("dialog", lambda d: d.accept())
page.get_by_role("button", name="Click for JS Confirm").click()
expect(page.locator("#result")).to_contain_text("Ok")
page.once("dialog", lambda d: d.dismiss())
page.get_by_role("button", name="Click for JS Confirm").click()
expect(page.locator("#result")).to_contain_text("Cancel")
```

বাংলা

অনুশীলন: (১) দুই context = দুই ব্যবহারকারী — একজন লগইন করলে অন্যজন লগইন-করা নয়, বিচ্ছিন্নতাই এখানে অ্যাসারশন; (২) Python দিয়ে ফাইল বানিয়ে আপলোড, পেজে নামের প্রতিধ্বনি যাচাই; (৩) `page.once("dialog", ...)` দিয়ে একবার accept, একবার dismiss — ফলের লেখা আলাদা হওয়া চাই।

Network Control & API Testing

নেটওয়ার্ক নিয়ন্ত্রণ ও API টেস্টিং

IN THIS CHAPTER · এই অধ্যায়ে

route(): intercept, mock, block, break · waiting for responses · APIRequestContext for pure API tests · the hybrid pattern (seed by API, verify by UI) · Lab: mock a flaky payment gateway.

4.1 route(): Sit Between the Page and the Internet

Plain-language first: `page.route()` lets your test intercept every request the page makes and decide its fate: let it through, block it, or answer it yourself with fake data.

PYTHON

```
# MOCK an API – the frontend can't tell the difference:
page.route("**/api/products", lambda route: route.fulfill(
    json=[{"sku": "R-77", "name": "Router AC1200", "price": 3850},
          {"sku": "C-06", "name": "Cat6 Cable", "price": 250}],
))

# BLOCK heavy assets – scraping/perf runs get 5x faster:
page.route("**/*.{png,jpg,jpeg,woff2}", lambda route: route.abort())

# BREAK the backend on purpose – test your error UI:
page.route("**/api/checkout", lambda route: route.fulfill(status=500))

page.goto("https://shop.example/catalog") # register routes BEFORE goto
```

Why mock in a UI test? **Determinism and reachability.** Real backends give different data every day (so assertions rot), and some states — payment declined, server on fire, empty catalog — are nearly impossible to produce on demand. With `route.fulfill()` you manufacture any state in one line and your assertions become exact.

বাংলা

`page.route()` আপনাকে পেজ আর ইন্টারনেটের মাঝখানে বসায়: প্রতিটি রিকোয়েস্ট যেতে দিন, আটকান (`abort`), বা নিজেই নকল উত্তর দিন (`fulfill`) — ফ্রন্টএন্ড পার্থক্য বোঝে না। কেন মক? **নিশ্চয়তা ও নাগাল:** আসল ব্যাকএন্ড রোজ ভিন্ন ডেটা দেয়, আর "পেমেন্ট ডিক্লাইন", "সার্ভার 500", "খালি ক্যাটালগ" — এসব অবস্থা চাহিবামাত্র তৈরি করা প্রায় অসম্ভব; মক দিয়ে এক লাইনে হয়। মনে রাখুন: রুট নিবন্ধন করুন `goto()`-র আগে।

4.2 Observing Traffic: expect_response

PYTHON

```
# Same arm-then-act idiom as downloads (3.3):
with page.expect_response("**/api/orders") as resp_info:
    page.get_by_role("button", name="Place order").click()
resp = resp_info.value
assert resp.status == 201
order_id = resp.json()["order_id"] # pivot: use API data in UI assertions
expect(page.get_by_text(f"Order {order_id} confirmed")).to_be_visible()
```

This checks *both layers* in one test: the API answered correctly (status, payload) **and** the UI rendered that answer correctly. When it fails, the failure tells you which layer broke — worth its weight in 3 AM debugging.

বাংলা

`expect_response()` সেই একই ছন্দ — শোতা আগে, ক্লিক পরে — কিন্তু এবার নেটওয়ার্ক উত্তরটি ধরা পড়ে। এক টেস্টে দুই স্তর যাচাই হয়: API সঠিক উত্তর দিয়েছে (status, payload) **এবং** UI সেই উত্তর সঠিকভাবে দেখিয়েছে। ফেল করলে বোঝা যায় কোন স্তরটি ভাঙল — রাত ৩টার ডিবাগিংয়ে এটি সোনার মতো দামি।

4.3 APIRequestContext: API Tests Without a Browser

Playwright ships an HTTP client too — same install, no browser window, perfect for pure API tests and for test setup:

PYTHON

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    api = p.request.new_context(base_url="https://api.frankfurter.dev")
    resp = api.get("/v1/latest", params={"base": "USD", "symbols": "BDT"})
    assert resp.ok # status 200-299
    rate = resp.json()["rates"]["BDT"]
    assert rate > 100 # sanity-check the payload
    print("USD->BDT:", rate)

# POST with a JSON body:
r = api.post("/v1/orders", data={"sku": "R-77", "qty": 2})
```

The **hybrid pattern** — the highest-leverage trick in this book: create test data through the API (fast, reliable), then verify behaviour through the UI (what users see). A UI test that first clicks through five screens to *create* its own test order is slow and triple-fragile.

বাংলা

Playwright-এর ভেতরেই একটি HTTP ক্লায়েন্ট আছে — ব্রাউজার ছাড়াই খাঁটি API টেস্ট: `api.get/post` → `resp.ok` → `resp.json()`। সবচেয়ে বড় কৌশল **হাইব্রিড প্যাটার্ন**: টেস্ট-ডেটা তৈরি করুন API দিয়ে (দ্রুত, নির্ভরযোগ্য), আর আচরণ যাচাই করুন UI দিয়ে (ব্যবহারকারী যা দেখে)। যে UI-টেস্ট নিজের ডেটা বানাতে পাঁচ স্ক্রিন ক্লিক করে, সেটি ধীর এবং তিনগুণ ভঙ্গুর।

4.4 Lab: Mock a Flaky Payment Gateway

Real scenario: the payment provider's sandbox times out randomly (the same flaky gateway from the Python Handbook's retry lab — now seen from the frontend). You must prove the UI handles *both* outcomes: success and failure. Mocking makes both deterministic:

PYTHON

```
# payment_lab.py – one page, two manufactured realities
from playwright.sync_api import sync_playwright, expect

PAGE = """
<button onclick="fetch('/api/pay',{method:'POST'})
  .then(r => r.ok ? r.json() : Promise.reject(r.status))
  .then(d => result.textContent = 'Paid: ' + d.txn_id)
  .catch(e => result.textContent = 'Payment failed – please retry')">Pay now</button>
<p id="result"></p>
"""

def run(page, *, gateway_up: bool):
    page.route("**/api/pay", lambda r: r.fulfill(json={"txn_id": "TXN-991"})
        if gateway_up else r.fulfill(status=503))
    page.set_content(PAGE) # tiny local page – no server needed
    page.get_by_role("button", name="Pay now").click()

with sync_playwright() as p:
    browser = p.chromium.launch()

    page = browser.new_context().new_page()
    run(page, gateway_up=True)
    expect(page.locator("#result")).to_have_text("Paid: TXN-991")
    print("✓ success path verified")

    page = browser.new_context().new_page()
    run(page, gateway_up=False) # the state you can't order from a real sandbox
    expect(page.locator("#result")).to_have_text("Payment failed – please retry")
    print("✓ failure path verified")
```

OUTPUT

```
✓ success path verified
✓ failure path verified
```

The failure path is the whole point: without mocking, you'd wait for the sandbox to *happen* to break. With `route.fulfill(status=503)` you order a broken gateway on demand and prove the error UI exists — most teams discover it doesn't.

বাংলা

বাস্তব দৃশ্য: পেমেন্ট স্যান্ডবক্স খামখেয়ালি — Python হ্যান্ডবুকের সেই flaky গेटওয়ে, এবার ফ্রন্টএন্ডের চোখে। UI-কে দুটি পরিণতিই সামলাতে হবে: সফল ও ব্যর্থ। মকিং দুটোকেই নিশ্চিত করে: `fulfill(json=...)` দিয়ে সাফল্য, `fulfill(status=503)` দিয়ে চাহিবামাত্র ভাঙা গेटওয়ে। ব্যর্থতার পথটিই আসল লাভ — মক ছাড়া আপনাকে অপেক্ষা করতে হতো কবে স্যান্ডবক্স নিজে ভাঙবে; আর বেশিরভাগ দল মক করেই আবিষ্কার করে যে তাদের এরর-UI আসলে নেই।

4.5 Common Mistakes

Mistake	Symptom	Fix
Registering <code>route()</code> after <code>goto()</code>	First-load requests slip through unmocked	Route first, navigate second
Glob doesn't match (<code>/api/pay</code> vs full URL)	Mock silently ignored	Use <code>**/api/pay</code> ; print <code>route.request.url</code> to debug

Mock shape drifts from the real API	Tests green, production broken	Copy payloads from DevTools; add one un-mocked smoke test
Asserting on network only	API fine, UI blank — test passes	Pair every <code>expect_response</code> with a visible-UI assertion
Creating test data by clicking through the UI	3-minute setup per test	Seed via <code>APIRequestContext</code> , verify via UI

4.6 Practice Exercises (with Answers)

- 1. Empty-state test:** mock `**/api/products` to return `[]` and assert the catalog shows "No products found".
- 2. Image blocker:** block all images on a heavy page and print how many requests were aborted (hint: a counter in the handler).
- 3. API contract check:** with `APIRequestContext`, GET the Frankfurter `/v1/currencies` endpoint and assert both that `resp.ok` and that `"BDT"` is a key.

PYTHON

```
# --- Answers (core lines) ---
# 1
page.route("**/api/products", lambda r: r.fulfill(json=[]))
page.goto(url)
expect(page.get_by_text("No products found")).to_be_visible()

# 2 — closures give the handler state:
blocked = {"n": 0}
def block(route):
    blocked["n"] += 1; route.abort()
page.route("**/*.png,*.jpg,*.gif", block)
page.goto(url); print("aborted:", blocked["n"])

# 3
resp = api.get("/v1/currencies")
assert resp.ok and "BDT" in resp.json()
```

বাংলা

অনুশীলন: (১) খালি ক্যাটালগ মক করে "No products found" যাচাই — খালি-অবস্থা টেস্টের ক্লাসিক; (২) ছবি ব্লক করে কতগুলো আটকালেন গুনুন — হ্যান্ডলারে ক্লোজার-কাউন্টার; (৩) ব্রাউজার ছাড়া খাঁটি API চেক: `resp.ok` এবং `"BDT"` কী-টি আছে কি না।

pytest Integration & Page Object Model

pytest ইন্টিগ্রেশন ও পেজ অবজেক্ট মডেল

IN THIS CHAPTER · এই অধ্যায়ে

pytest-playwright fixtures · running across browsers · conftest & auth fixtures · parametrize · Page Objects done right · Lab: refactor a raw suite into POM.

5.1 pytest-playwright: The Boilerplate Disappears

Remember `sync_playwright() → launch → new_context → new_page → close`? The plugin you installed in 0.3 does all of it. Ask for a `page` argument and you receive a fresh page in a fresh context, cleaned up automatically — one isolated context per test, exactly the 3.1 rule:

PYTHON

```
# test_smoke.py – a complete, real test file
import re
from playwright.sync_api import Page, expect

def test_homepage_title(page: Page):
    page.goto("https://playwright.dev/python")
    expect(page).to_have_title(re.compile("Playwright"))

def test_docs_link(page: Page):
    page.goto("https://playwright.dev/python")
    page.get_by_role("link", name="Docs").click()
    expect(page).to_have_url(re.compile(r"/docs/intro"))
```

TERMINAL

```
pytest -q # headless chromium: 2 passed
pytest --headed --slowmo 300 # watch it, human speed
pytest --browser firefox --browser webkit # same tests, other engines
pytest --tracing retain-on-failure # trace file for every failure (6.1)
```

বাংলা

0.3-এ ইনস্টল করা প্লাগইনটিই সব বয়লারপ্লেট সরিয়ে দেয়: টেস্ট ফাংশনে `page` আর্গুমেন্ট চাইলেই নতুন context-এ নতুন পেজ পাবেন, শেষে নিজে থেকেই পরিষ্কার — প্রতি টেস্টে বিচ্ছিন্ন context, ঠিক ৩.১-এর নিয়ম। কমান্ড-লাইন থেকেই ক্ষমতা: `--headed` দেখে দেখে, `--browser firefox` অন্য ইঞ্জিনে, `--tracing retain-on-failure` ফেল হলে ট্রেস।

5.2 conftest.py: Auth Fixture & Parametrize

The 3.5 login-once pattern becomes a fixture: session-scoped login, per-test authenticated pages. Every test that asks for `auth_page` starts logged in:

PYTHON

```
# conftest.py – shared plumbing for the whole suite
import pytest
from playwright.sync_api import expect

BASE = "https://www.saucedemo.com"

@pytest.fixture(scope="session")
def auth_state(browser, tmp_path_factory):
    """UI login ONCE per run; hand every test the saved state."""
    path = tmp_path_factory.mktemp("auth") / "state.json"
    page = browser.new_page()
    page.goto(BASE)
    page.get_by_placeholder("Username").fill("standard_user")
    page.get_by_placeholder("Password").fill("secret_sauce")
    page.get_by_role("button", name="Login").click()
    expect(page).to_have_url("**/inventory.html")
    page.context.storage_state(path=path)
    page.close()
    return path

@pytest.fixture
def auth_page(browser, auth_state):
    ctx = browser.new_context(storage_state=auth_state)
    yield ctx.new_page()          # test runs here
    ctx.close()                  # teardown always runs

# test_inventory.py – parametrize multiplies coverage for free
import pytest
from playwright.sync_api import expect

@pytest.mark.parametrize("option,first_item", [
    ("lohi", "Sauce Labs Onesie"),
    ("hilo", "Fleece Jacket"),
    ("za", "Test.allTheThings() T-Shirt (Red)"),
])
def test_sorting(auth_page, option, first_item):
    auth_page.goto(f"{BASE}/inventory.html")
    auth_page.locator("[data-test=product-sort-container]").select_option(option)
    expect(auth_page.locator(".inventory_item_name").first).to_contain_text(first_item)
```

বাংলা

৩.৫-এর "একবার লগইন" প্যাটার্নটি ফিক্সচারে রূপ নেয়: `scope="session"` মানে পুরো রানে একবারই UI-লগইন; প্রতিটি টেস্ট `auth_page` চাইলেই লগইন-করা নতুন পেজ পায়, `yield`-এর পরের অংশ টিয়ারডাউন হিসেবে সবসময় চলে। আর `@pytest.mark.parametrize` একই টেস্ট-দেহকে ভিন্ন ডেটায় বারবার চালায় — এক ফাংশনে তিনটি সাজানোর নিয়ম যাচাই, বিনা খরচে কভারেজ তিনগুণ।

5.3 Page Object Model: One Class per Screen

Plain-language first: when a button's label changes, do you edit one file or forty tests? A Page Object is a class that owns one screen's locators and actions, so page knowledge lives in exactly one place.

PYTHON

```
# pages/inventory_page.py
from playwright.sync_api import Page

class InventoryPage:
    URL = "https://www.saucedemo.com/inventory.html"

    def __init__(self, page: Page):
        self.page = page
        self.sort_select = page.locator("[data-test=product-sort-container]")
        self.cart_badge = page.locator(".shopping_cart_badge")
        self.item_names = page.locator(".inventory_item_name") # locators are lazy (1.1) –
                                                                # safe to build in __init__

    def open(self):
        self.page.goto(self.URL)

    def sort_by(self, option: str):
        self.sort_select.select_option(option)

    def add_to_cart(self, product: str):
        row = self.page.locator(".inventory_item").filter(has_text=product)
        row.get_by_role("button", name="Add to cart").click()

# test_cart.py – the test now reads like the business requirement
def test_add_two_products(auth_page):
    inventory = InventoryPage(auth_page)
    inventory.open()
    inventory.add_to_cart("Sauce Labs Backpack")
    inventory.add_to_cart("Sauce Labs Bike Light")
    expect(inventory.cart_badge).to_have_text("2")
```

★ KEY POINT

Rules that keep POM clean: page objects hold **locators and actions**, **never assertions** (tests assert); methods are named after **user intent** (`add_to_cart` , not `click_third_button`); and expose locators as attributes so tests can `expect()` them directly.

বাংলা

বোতামের লেবেল বদলালে এক ফাইল বদলাবেন, না চল্লিশটি টেস্ট? Page Object হলো এক স্ক্রিনের লোকেটর ও অ্যাকশনের মালিক এক ক্লাস — পেজ-জ্ঞান ঠিক এক জায়গায়। নিয়ম: পেজ অবজেক্টে **লোকেটর ও অ্যাকশন, কখনো অ্যাসারশন নয়** (অ্যাসার্ট করবে টেস্ট); মেথডের নাম ব্যবহারকারীর উদ্দেশ্যে (`add_to_cart`); লোকেটরগুলো অ্যাট্রিবিউট হিসেবে থোলা রাখুন যেন টেস্ট সরাসরি `expect()` করতে পারে। লোকেটর অলস (১.১) বলেই `__init__` -এ বানানো নিরাপদ।

5.4 Lab: Refactor a Raw Suite into POM

Real scenario: your team's checkout flow test is 40 lines of raw locators repeated across 6 files. Restructure into two page objects plus one readable test:

PYTHON

```
# pages/checkout_page.py
class CheckoutPage:
    def __init__(self, page):
        self.page = page
        self.total = page.locator(".summary_total_label")
        self.complete_header = page.locator(".complete-header")

    def start_from_cart(self):
        self.page.locator(".shopping_cart_link").click()
        self.page.get_by_role("button", name="Checkout").click()

    def fill_shipping(self, first, last, zip_code):
        self.page.get_by_placeholder("First Name").fill(first)
        self.page.get_by_placeholder("Last Name").fill(last)
        self.page.get_by_placeholder("Zip/Postal Code").fill(zip_code)
        self.page.get_by_role("button", name="Continue").click()

    def finish(self):
        self.page.get_by_role("button", name="Finish").click()

# test_checkout.py — six screens, told as a story
def test_full_checkout(auth_page):
    inventory = InventoryPage(auth_page)
    checkout = CheckoutPage(auth_page)

    inventory.open()
    inventory.add_to_cart("Sauce Labs Backpack")
    checkout.start_from_cart()
    checkout.fill_shipping("Rahim", "Uddin", "1207")
    expect(checkout.total).to_contain_text("$32.39") # price + tax
    checkout.finish()
    expect(checkout.complete_header).to_have_text("Thank you for your order!")
```

TERMINAL

```
pytest -q
.... [100%]
4 passed in 9.31s
```

Read the test aloud — it is the acceptance criterion, verbatim. When the app's markup changes, only [pages/](#) changes; the tests, which encode *business rules*, stay untouched. That separation is what lets suites survive years of redesigns.

বাংলা

বাস্তব দৃশ্য: চেকআউট ফ্লো-র ৪০ লাইনের কাঁচা লোকেটর ৬টি ফাইলে কপি হয়ে আছে। দুটি পেজ অবজেক্ট + একটি পাঠযোগ্য টেস্টে সাজান। টেস্টটি জোরে পড়ুন — এটি হুবহু acceptance criterion! অ্যাপের মার্কআপ বদলালে বদলাবে শুধু [pages/](#) ; ব্যবসায়িক নিয়ম ধারণকারী টেস্টগুলো অক্ষত — এই বিচ্ছেদই স্যুটকে বছরের পর বছর রিডিজাইন সয়ে টিকিয়ে রাখে।

5.5 Common Mistakes

Mistake	Symptom	Fix
Assertions inside page objects	Same class asserts differently per test's needs	Pages expose locators; tests <code>expect()</code>

One test depends on another's cart	Passes alone, fails in suite (or reverse)	Independent tests; fixtures build state
Login UI steps repeated per test	Slow suite	Session-scoped <code>auth_state</code> fixture
God-object <code>BasePage</code> with 80 helpers	Nobody knows where anything lives	One class per screen; helpers only when shared 3+ times
Test names like <code>test_1</code>	Failure list is unreadable	<code>test_sorting_low_to_high_shows_cheapest_first</code>
Fixtures that assert	Setup failures masquerade as test failures	Fixtures arrange; tests assert

5.6 Practice Exercises (with Answers)

- LoginPage object:** write `LoginPage` with a `login(user, pw)` method and an `error` locator; test both a good and a bad password using it.
- Parametrized users:** saucedemo ships `problem_user` and `performance_glitch_user`. Parametrize the login test across all three users.
- Fixture teardown proof:** add a `print` after `yield` in `auth_page` and run `pytest -s` to watch teardown fire even when a test fails.

PYTHON

```
# --- Answer 1 (the pattern the others reuse) ---
class LoginPage:
    def __init__(self, page):
        self.page = page
        self.error = page.locator("[data-test=error]")
    def open(self): self.page.goto("https://www.saucedemo.com")
    def login(self, user, pw):
        self.page.get_by_placeholder("Username").fill(user)
        self.page.get_by_placeholder("Password").fill(pw)
        self.page.get_by_role("button", name="Login").click()

def test_wrong_password_shows_error(page):
    lp = LoginPage(page); lp.open()
    lp.login("standard_user", "wrong")
    expect(lp.error).to_contain_text("do not match")
```

বাংলা

অনুশীলন: (১) `LoginPage` — `login()` মেথড ও `error` লোকেটর; সঠিক ও ভুল পাসওয়ার্ড দুটোই টেস্ট করুন; (২) তিন ব্যবহারকারীতে (`standard/problem/performance_glitch`) লগইন parametrize করুন; (৩) `yield`-এর পরে `print` বসিয়ে `pytest -s` চালান — টেস্ট ফেল করলেও টিয়ারডাউন চলে, নিজের চোখে দেখুন।

Production: Tracing, Parallelism, CI & Debugging

প্রোডাকশন: ট্রেসিং, প্যারালাল, CI ও ডিবাগিং

IN THIS CHAPTER · এই অধ্যায়ে

The trace viewer (your flight recorder) · screenshots & video on failure · parallel workers & retries · GitHub Actions pipeline · debugging tools · Capstone: e-commerce smoke suite.

6.1 Tracing: The Flight Recorder for Failed Tests

Plain-language first: a trace is a recording of everything a test did — DOM snapshots before/after each action, network calls, console logs — replayable in a time-travel viewer. When CI fails at 3 AM, the trace is the only witness.

TERMINAL

```
pytest --tracing retain-on-failure          # record everything, keep only failures
playwright show-trace test-results/<test-name>/trace.zip
pytest --screenshot only-on-failure --video retain-on-failure # lighter artifacts
```

In the viewer you scrub a timeline of the run, click any action, and see the page *exactly as the browser saw it* at that instant, plus which locator was used and what the network was doing. It converts "cannot reproduce" into "oh — the coupon banner covered the button." Make `retain-on-failure` your permanent default; the overhead is negligible next to one saved debugging hour.

বাংলা

ট্রেস হলো টেস্টের ব্ল্যাকবক্স: প্রতিটি অ্যাকশনের আগে-পরে DOM-স্ন্যাপশট, নেটওয়ার্ক কল, কনসোল লগ — টাইম-ট্রাভেল ভিউয়ারে চালিয়ে দেখা যায়। CI-তে রাত ৩টায় ফেল করলে ট্রেসই একমাত্র সাক্ষী: টাইমলাইন টেনে যেকোনো মুহুর্তে পেজ ঠিক যেমন ছিল তেমনটি দেখুন। "রিপ্রোডিউস হয় না"-কে এটি বদলে দেয় "ও — কুপন ব্যানারটা বোতাম ঢেকে রেখেছিল"-তে। `--tracing retain-on-failure` স্থায়ী ডিফল্ট করে নিন।

6.2 Parallel Workers, Retries & Cross-Browser

TERMINAL

```
pip install pytest-xdist
pytest -n 4                      # 4 workers — near-4x faster wall-clock
pip install pytest-rerunfailures
pytest --reruns 1 --reruns-delay 2 # retry genuine infra blips once
pytest --browser chromium --browser firefox --browser webkit -n 6
```

Parallelism only works because of the discipline you already built: independent tests, own contexts, no shared state (5.5). Retries are for *infrastructure* flakiness (network blips, cold CDN) — a test that needs retries to pass on a healthy system is a bug; fix the wait, don't hide it.

বাংলা

`pytest -n 4` দিয়ে ৪টি ওয়ার্কারে সমান্তরাল রান — প্রায় ৪ গুণ দ্রুত। এটি সম্ভব শুধু আগের শৃঙ্খলার কারণেই: স্বাধীন টেস্ট, নিজস্ব context, ভাগাভাগি করা state নেই। রিট্রাই (`--reruns 1`) কেবল অবকাঠামোগত সমস্যার জন্য — সুস্থ সিস্টেমে যে টেস্ট পাস করতে রিট্রাই লাগে, সেটি নিজেই একটি বাগ: অপেক্ষাটি ঠিক করুন, লুকানো না।

6.3 CI: The Suite Runs Itself (GitHub Actions)

YAML

```
# .github/workflows/e2e.yml – every push, every night
name: e2e
on: [push, pull_request, workflow_dispatch]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12"}
      - run: pip install -r requirements.txt
      - run: playwright install --with-deps chromium
      - run: pytest -n 4 --tracing retain-on-failure
    env:
      APP_PASSWORD: ${ secrets.APP_PASSWORD } # secrets, never in code (7.x rule)
      - uses: actions/upload-artifact@v4
        if: failure() # publish traces when red
        with: {name: traces, path: test-results/}
```

The workflow encodes the whole book: install, browsers with system deps, parallel run, traces kept on failure and uploaded where humans can download them, and credentials via CI secrets — never the repository. Exit code discipline comes free: pytest returns non-zero on failure, so the pipeline goes red and alerts.

বাংলা

CI ওয়ার্কফ্লোতে পুরো বইটি ধরা আছে: ইনস্টল → `playwright install --with-deps` → সমান্তরাল রান → ফেল হলে ট্রেস আর্টিফ্যাক্ট হিসেবে আপলোড → পাসওয়ার্ড CI secrets-এ (কোডে কখনো নয়)। `pytest` ব্যর্থতায় non-zero exit code দেয়, তাই পাইপলাইন নিজেই লাল হয়ে সতর্ক করে — Python হ্যান্ডবুকের সেই exit-code শৃঙ্খলা এখানে বিনামূল্যে মেলে।

6.4 Debugging Toolbox

TERMINAL

```
PWDEBUG=1 pytest -k test_checkout # Inspector: step through, live-edit locators
pytest --headed --slowmo 500      # watch at human speed
```

PYTHON

```
page.pause() # breakpoint that opens the Inspector right here
print(page.get_by_role("button").all_inner_texts()) # what CAN I click?
page.screenshot(path="wtf.png", full_page=True) # eyes on the state
```

Escalation path when a locator won't cooperate: run `--headed` → drop a `page.pause()` just before the failure → use the Inspector's "Pick locator" to test candidates live → if it only fails in CI, open the trace. In the Inspector you can edit a locator and watch it highlight matches in real time — the fastest locator feedback loop that exists.

বাংলা

ডিবাগ-সিঁড়ি: `--headed` -এ চালান → ব্যর্থতার ঠিক আগে `page.pause()` বসান (Inspector খুলে যায়) → "Pick locator" দিয়ে লোকেটর লাইভ পরীক্ষা করুন → কেবল CI-তে ফেল করলে ট্রেস খুলুন। Inspector-এ লোকেটর লিখলেই পেজে মিলগুলো সাথে সাথে হাইলাইট হয় — লোকেটর শেখার দ্রুততম ফিডব্যাক লুপ এটিই।

6.5 Capstone Lab: E-Commerce Smoke Suite, CI-Ready

Real scenario: management wants a nightly smoke suite answering one question — *"can a customer still buy?"* — cross-browser, parallel, with traces on failure. One tree, every module on duty:

OUTPUT

```
shop-e2e/
├─ conftest.py          # auth_state + auth_page fixtures (5.2)
├─ pages/
│  └─ login_page.py     # POM (5.3)
│  └─ inventory_page.py
│  └─ checkout_page.py
├─ tests/
│  └─ test_login.py      # incl. wrong-password path
│  └─ test_cart.py
│  └─ test_checkout.py   # the money path (5.4)
│  └─ test_api_health.py # APIRequestContext canary (4.3)
├─ requirements.txt
└─ .github/workflows/e2e.yml
```

PYTHON

```
# tests/test_checkout.py – the one test that guards revenue
def test_customer_can_buy(auth_page):
    inventory = InventoryPage(auth_page)
    checkout = CheckoutPage(auth_page)

    inventory.open()
    inventory.add_to_cart("Sauce Labs Backpack")
    expect(inventory.cart_badge).to_have_text("1")           # checkpoint 1

    checkout.start_from_cart()
    checkout.fill_shipping("Rahim", "Uddin", "1207")
    expect(checkout.total).to_contain_text("$32.39")         # checkpoint 2: money is right

    checkout.finish()
    expect(checkout.complete_header).to_have_text("Thank you for your order!")
    expect(auth_page).to_have_url("**/checkout-complete.html")
```

TERMINAL

```
pytest -n 4 --browser chromium --browser firefox --tracing retain-on-failure
.....                                     [100%]
16 passed in 21.44s
```

The anatomy of a production suite: business flows as readable tests; page knowledge quarantined in `pages/`; login paid once per session; an API canary that fails fast when the backend itself is down (so UI failures aren't misblamed); parallel workers; and traces published by CI. Extend it: a mocked payment-failure test (4.4), a `--base-url` option for staging vs production, a nightly `schedule:` trigger.

বাংলা

ক্যাপস্টোন দৃশ্য: ম্যানেজমেন্ট চায় প্রতি রাতে এক প্রশ্নের উত্তর — *"ক্রেতা কি এখনো কিনতে পারছে?"* প্রোডাকশন স্যুটের শারীরস্থান দেখুন: ব্যবসায়িক ফ্লো = পাঠযোগ্য টেস্ট; পেজ-জ্ঞান `pages/` -এ বন্দি; লগইনের খরচ প্রতি রানে একবার; ব্যাকএন্ড নিজেই পড়ে গেলে দ্রুত জানান দিতে একটি API-ক্যানারি; সমান্তরাল ওয়ার্কার; ফেল হলে CI ট্রেস প্রকাশ করে। নিজে বাড়ান: ৪.৪-এর মক-করা পেমেন্ট-ব্যর্থতা টেস্ট, স্টেজিং/প্রোডাকশনের জন্য `--base-url`, রাতিকালীন `schedule` ট্রিগার।

6.6 Production Checklist

Before a suite guards a real release pipeline, walk this list:

- ✓ Zero `sleep()` / `wait_for_timeout()`; every wait is an action or an `expect()`.
- ✓ User-facing locators throughout; raw CSS/XPath only with a comment explaining why.
- ✓ Every test runs alone *and* shuffled (`pytest -p no:randomly` off); no ordering coupling.
- ✓ Login via `storage_state` fixture; credentials from env vars/CI secrets, never code or git.
- ✓ Unreachable states (payment declined, 500s, empty lists) covered via `route()` mocks.
- ✓ `--tracing retain-on-failure` on by default; CI uploads traces as artifacts.
- ✓ Parallel-safe (`-n 4`) and green cross-browser at least on Chromium + one other engine.
- ✓ A README line says how to install (`pip install -r requirements.txt; playwright install`) and run.

বাংলা

প্রোডাকশন চেকলিস্ট: কোনো `sleep` নেই — প্রতিটি অপেক্ষা অ্যাকশন বা `expect()`; সর্বত্র ব্যবহারকারী-মুখী লোকেটর; প্রতিটি টেস্ট একা ও এলোমেলো ক্রমে সবুজ; লগইন `storage_state` ফিক্সচারে আর গোপন তথ্য env/CI secrets-এ; অসম্ভব অবস্থাগুলো mock-এ ঢাকা; ফেল হলে ট্রেস; সমান্তরাল-নিরাপদ ও অন্তত দুই ব্রাউজারে সবুজ; ইনস্টল-রান নির্দেশনা README-তে। এই তালিকা পাস করলে স্যুট সত্যিই রিলিজ পাহারা দেওয়ার যোগ্য।

Cheat Sheet, Error Decoder & Next Steps

চিটশিট, এরর ডিকোডার ও পরবর্তী ধাপ

IN THIS CHAPTER · এই অধ্যায়ে

One-page syntax cheat sheet · a decoder table for the 10 errors you will actually meet · ten graded real projects
· the roadmap after this book.

A.1 One-Page Cheat Sheet

PYTHON

```
# ---- boot ----
from playwright.sync_api import sync_playwright, expect
with sync_playwright() as p:
    page = p.chromium.launch(headless=False).new_context().new_page()
# ---- locators (in priority order) ----
page.get_by_role("button", name="Login")    page.get_by_label("Email")
page.get_by_placeholder("Search")    page.get_by_text("Done", exact=True)
page.get_by_test_id("total")    page.locator("css")    # last resort
loc.filter(has_text="Backpack")    parent.get_by_role(...)    # chain
loc.first    loc.nth(2)    loc.count()    loc.all_inner_texts()
# ---- actions (all auto-wait) ----
loc.fill(v)    loc.click()    loc.check()    loc.select_option(v)
loc.press("Enter")    loc.hover()    loc.set_input_files(path)
# ---- assertions (all retry) ----
expect(loc).to_be_visible()    .to_have_text(t)    .to_contain_text(t)
expect(loc).to_have_count(n)    .to_be_enabled()    .to_have_value(v)
expect(page).to_have_url("**/done")    expect(loc).not_to_be_visible()
# ---- state / awkward pages ----
context.storage_state(path="auth.json")
browser.new_context(storage_state="auth.json")
with page.expect_download() as d: ...    page.once("dialog", lambda d: d.accept())
with context.expect_page() as np: ...    page.frame_locator("#pay-frame").get_by_label(...)
# ---- network ----
page.route("**/api/x", lambda r: r.fulfill(json=[...]))    # mock
page.route("**/*.png", lambda r: r.abort())    # block
with page.expect_response("**/api/x") as r: ...
api = p.request.new_context(base_url=U); api.get("/v1/x").json()
# ---- pytest ----
def test_x(page): ...    # fixture: fresh context per test
# pytest --headed --browser firefox -n 4 --tracing retain-on-failure
# PWDEBUG=1 pytest -k name | page.pause() | playwright codegen URL
```

বাংলা

এক পাতার চিটশিট — বইয়ের প্রতিটি মূল অঙ্ক এক জায়গায়: লোকেটর-অগ্রাধিকার, অটো-ওয়েটিং অ্যাকশন, রিট্রাই-করা `expect()`, `storage_state`, ডাউনলোড/ডায়ালগ/iframe-এর ছন্দ, `route()` মকিং, API ক্লায়েন্ট, আর pytest কমান্ডগুলো। প্রিন্ট করে ডেস্কে রাখুন।

A.2 Error Message Decoder

Error	Usual meaning	First thing to check
<code>TimeoutError: waiting for locator(...)</code>	Locator never matched / never actionable	Typo in name; element in an iframe? Run headed, <code>page.pause()</code>
<code>strict mode violation: resolved to 3 elements</code>	Locator is ambiguous	Refine with <code>filter()</code> / roles, don't reflex- <code>.first</code>
<code>Element is not visible / intercepts pointer events</code>	Covered by overlay/cookie banner	Close the overlay; never blanket <code>force=True</code>
<code>Target page/context/browser has been closed</code>	Using a page after teardown, or crashed	Fixture scopes; anything after <code>ctx.close()</code> ?
<code>Executable doesn't exist ... run playwright install</code>	Browser binaries missing in this env	<code>playwright install chromium</code> (in CI: <code>--with-deps</code>)
<code>net::ERR_NAME_NOT_RESOLVED</code>	Bad URL / no network / typo in mock glob	Print the URL; check <code>base_url</code> config
<code>expect(...).to_have_text: expected ... received ...</code>	Real mismatch, or trailing whitespace/casing	Read the diff; try <code>to_contain_text</code>
<code>Error: page.goto: navigation interrupted</code>	App redirected mid-load (auth wall?)	Is <code>auth.json</code> stale? Re-run the login script
<code>ImportError: cannot import name 'sync_playwright'</code>	You have a file shadowing the library	Rename your <code>playwright.py</code>
Async-loop errors mentioning <code>asyncio</code>	Sync API inside Jupyter/async app	Use the async API there, or plain scripts

Read Playwright failures top-down for the summary, then the **call log** at the bottom — it lists each actionability check and shows exactly which one was still failing when time ran out.

বাংলা

এরর ডিকোডার: `TimeoutError` = লোকেটর মেলেনি (বানান? iframe?); `strict mode violation` = লোকেটর অস্পষ্ট — নিখুঁত করুন; "intercepts pointer events" = কিছু একটা ঢেকে আছে (কুকি ব্যানার!); `Target closed` = বন্ধ হয়ে যাওয়া পেজ ব্যবহার করছেন; `Executable doesn't exist` = `playwright install` চালান। ব্যর্থতার নিচের **call log**-টি পড়ুন — কোন actionability-চেকটি আটকে ছিল তা সরাসরি লেখা থাকে।

A.3 Ten Real Projects to Build Next

Ordered by difficulty; each names the modules it exercises:

1. **Form filler** — automate a public practice form end to end, assert the success screen (1-2).
2. **Price watcher** — scrape one product's price into CSV daily; alert (print) below a threshold (1-2 + your Python skills).
3. **Screenshot service** — CLI: `python shot.py URL out.png` with full-page and mobile-viewport flags (1, argparse).
4. **Login smoke pack** — good + bad + locked-out credentials on a demo site, POM-structured (2, 3, 5).
5. **Session vault** — script that refreshes `auth.json` for three demo users and verifies each (3).
6. **Dead-link crawler** — collect every `<a href>` on your site, check each with `APIRequestContext`, report 404s (1, 4).

7. **Empty/error-state gallery** — mock a real SPA's APIs into empty, slow and 500 states; screenshot each (4).
8. **Multi-user chat test** — two contexts talking to each other in one test: A sends, B sees (3).
9. **Nightly regression in CI** — your Module-06 capstone on a schedule, traces uploaded, Slack/email on red (5–6).
10. **Visual diff harness** — screenshot key pages per commit and diff against baselines with Pillow (6 + Python Handbook 7).

Rule: build without looking at the book first; open it only when stuck 20+ minutes.

বাংলা

পরবর্তী দশ প্রজেক্ট (সহজ → কঠিন): ফর্ম-ফিলার, প্রাইস-ওয়াচার, স্ক্রিনশট CLI, লগইন স্মোক-প্যাক, সেশন-ভল্ট, ডেড-লিংক ক্রলার, খালি/এরর-স্টেট গ্যালারি, দুই-ব্যবহারকারী চ্যাট টেস্ট, CI-তে নৈশ রিগ্রেশন, ভিজুয়াল ডিফ। নিয়ম: আগে বই না দেখে বানান; ২০ মিনিট আটকে থাকলে তবেই খুলুন।

A.4 Where to Go Next

- ✓ **Official docs** — playwright.dev/python: the reference you'll use forever; the Node docs' concepts transfer 1:1.
- ✓ **Async API** — same methods with `await`; needed inside FastAPI apps, Jupyter, or thousand-page scraping fleets.
- ✓ **Deeper testing craft** — accessibility checks (axe), visual regression, contract tests; read "Testing" patterns beyond tooling.
- ✓ **Scraping at scale** — politeness (rate limits, robots.txt), rotating contexts, `route()`-blocking assets for speed.
- ✓ **Quality bar** — run `ruff` and `mypy` on your suite like any production code; type-hint your page objects.

Keep the loop from 0.5: **automate → watch it fail → read the trace → fix**. That loop, repeated for months, is the entire secret.

বাংলা

এরপরের পথ: অফিসিয়াল ডকুমেন্টেশন সারাজীবনের রেফারেন্স; বড় স্কেল বা FastAPI/Jupyter-এ async API; এরপর অ্যাক্সেসিবিলিটি ও ভিজুয়াল-রিগ্রেশন টেস্টিং; স্ক্র্যাপিংয়ে ভদ্রতা (রেট-লিমিট, robots.txt) মেনে চলুন; আর টেস্ট-কোডেও `ruff` ও `mypy` চালান। ০.৫-এর চক্রটাই আসল রহস্য: **অটোমেট করুন → ফেল দেখুন → ট্রেস পড়ুন → ঠিক করুন** — মাসের পর মাস।